

R2 LogiFlow Prep

All likely LogiFlow questions for GEP Round 2 · Ojas Srivastava

R2 goes deeper on projects. LogiFlow is where they will spend the most time. Know every number.

From reports: GFG SDE: R2 = project ML deep-dive + Fibonacci two ways. GFG Intern: R2 “Explain LogiFlow to a product manager.” GFG ASD: R2 = projects + OOP one-liners + SQL. Shivam Kumar (SDE 2025): questions from intro/resume, not a bank.

R2 context. They already heard your intro and OOP in R1. R2 opens with a short intro (30 seconds), then goes functional/product on LogiFlow. They want to know: do you **own** this, or did you just show up?

Your 30-second R2 intro:

Good afternoon. I am Ojas Srivastava, pre-final year B.Tech AI at SVNIT Surat, 9.20 CGPA. I actively compete in hackathons. Technical co-lead on LogiFlow — Google Solution Challenge Global Top 106. Solo-built Community Hero — Vibe2Ship Global Top 20. Chairperson of Nexus — I lead a team of sixteen and we manage initiatives for 500-plus students.

1. The product pitch (3–20 min of R2)

Q. Explain LogiFlow to a product manager. Not code — the product.

Typical: “It is an AI logistics app we made for a hackathon.”

You say. A planner has freight to move. They open LogiFlow, set constraints — budget ceiling, delivery window, acceptable delay risk — and get ranked corridors. Each corridor shows time, cost, and a delay-risk score. They pick a route and commit. The product is the **ranking**, not the map. Without it, the planner calls three brokers and guesses.

Q. Who uses it? What is the user journey?

Typical: “Everyone in logistics.”

You say. A logistics operator or procurement planner at a mid-size shipper. Journey: log in → enter origin, destination, constraints → see ranked corridors on a map → compare trade-offs (fast but expensive vs slow but cheap) → select and commit. The comparator pipeline exists so they see trade-offs, not one answer.

Q. What breaks? What is the worst failure?

Typical: “Nothing, it works.”

You say. Worst case: stale ranking leads to a wrong commit. Mitigation: Redis TTL + SQL as source of truth. If cache returns a corridor that no longer exists in SQL, the commit step validates before confirming. Second failure: model gives a bad delay estimate. Mitigation: delay risk is one input to ranking, not the only signal — time and cost still hold. I surface the confidence, not hide it.

Q. What would you change if you had another month?

Typical: “Add more AI.”

You say. Three things. First, user-configurable weights on time vs cost vs delay — right now the ranking is fixed. Second, historical commit data to learn which corridors planners actually pick and bias toward those. Third, alerting — if a corridor’s delay risk spikes after a commit, notify the planner before the shipment moves.

2. Architecture and ownership

Q. Draw the architecture. What calls what?

Typical: “Frontend talks to backend.”

You say. Next.js UI → FastAPI → cache-aside on Redis (check → miss → SQL → pipeline → rank → SET with TTL → return). Three pipelines feed the ranker: railway, comparator, hybrid. pytest runs in CI before merge. GCP Cloud Run hosts both API and UI.

Q. What was your code vs the team's?

Typical: “I did everything / I was the main lead.”

You say. Co-lead. I owned: railway pipeline (geometry, reads, ranking into API, tests), comparator pipeline (compare modes so user sees trade-offs), hybrid pipeline, and the test suite. Frontend was shared — I do not claim I wrote every React file. Data ingestion was shared. I can walk you through any file I wrote.

Q. How many people on the team? How did you split work?

Typical: “Four people, we all did everything.”

You say. Four. I took the backend pipelines and tests. One person owned data ingestion and cleaning. One owned the Next.js UI and map rendering. One handled deployment and Cloud Run config. We used GitHub PRs — every merge was reviewed. Co-lead means I also reviewed others' code and made architecture calls on the ranking logic.

Q. Show me the code for ranking. How does it work?

Typical: “It is a neural network.”

You say. Not a neural network. Scoring function: each corridor gets a composite score from three normalised dimensions — time, cost, delay risk. Pareto-style: I do not hide a slow cheap route behind a fast expensive one. The user sees the frontier. Delay risk comes from a gradient-boosting model (sklearn, not PyTorch) trained on 15k+ train-day records — MAE around 22 minutes. The model is one input; the ranking logic is deterministic.

3. Redis and caching (they always ask this)

Q. Why Redis? Why not just SQL?

Typical: “Redis is fast.”

You say. Corridor metadata is read far more than written — classic read-heavy pattern. SQL is the source of truth but recomputing rankings on every request took seconds. Redis gives sub-millisecond lookups. Cache-aside: check Redis → miss → compute from SQL → SET with TTL → return. Reads drop to 100–400 ms.

Q. What if Redis is down?

Typical: “The app crashes.”

You say. FastAPI catches the Redis connection error and falls back to SQL. Slower, but correct. Redis is never the source of truth. I do not serve a 500 because the cache is unavailable.

Q. What if the cache is stale?

Typical: “We invalidate on every write.”

You say. TTL handles most staleness — corridor data does not change per request. On write (new tariff, corridor update), I invalidate the affected keys. Stale reads inside TTL are acceptable for map display — a planner seeing a 2-minute-old ranking is fine. Not acceptable for an irreversible commit — the commit step re-validates against SQL.

Q. What TTL did you use? How did you pick it?

Typical: “I don't remember.”

You say. TTL depends on how often the underlying data changes. For corridor metadata that updates a few times a day, a TTL of 5–15 minutes is reasonable. I would not cache with a 24-hour TTL because tariff changes would be invisible. I would not cache with a 10-second TTL because that defeats the purpose. The right TTL is a product decision, not just a tech one.

4. Data and pipelines

Q. Where does the data come from? 15,000 train-day records?

Typical: “We used an API.”

You say. Indian Railways public data — schedules, routes, historical delays. 15,000+ train-day records across 580+ corridors. We ingested, cleaned, joined, and validated in SQL/Python pipelines. No live API call per request — batch ingest, then serve from the processed store.

Q. What is a “corridor”?

Typical: “A route.”

You say. A corridor is a logical freight path between two points — it may involve multiple legs (rail, road, or both). Each corridor has attributes: distance, estimated time, cost, and a delay-risk score. 580+ corridors means 580+ distinct origin-destination pairs with at least one viable path.

Q. What is the railway pipeline vs comparator vs hybrid?

Typical: “Three different projects.”

You say. Three stages of the same system. Railway pipeline: processes rail-only corridors — geometry, schedule data, delay model. Comparator pipeline: compares rail vs road for the same OD pair so the user sees trade-offs. Hybrid pipeline: multi-modal corridors — rail for trunk, road for first/last mile. All three feed the same ranker interface.

5. Testing

Q. 100/100 pytest — what are you testing?

Typical: “We test everything.”

You say. Business rules, not UI clicks. Pricing rules: does a tariff change propagate correctly to the ranking? Routing rules: does a blocked corridor get excluded? Edge cases: zero-distance corridor, negative cost, missing delay data. Each test is a contract — if this input, then this output. If a test fails, CI blocks the merge.

Q. How do you test Redis caching?

Typical: “We don’t.”

You say. Integration test: seed Redis with known data, hit the API, assert the response matches. Then invalidate, hit again, assert it recomputes from SQL. Also test the fallback: mock Redis as unavailable, assert the API still returns correct data from SQL, just slower.

6. Connecting to GEP

Q. This is a hackathon project. How is it relevant to GEP?

Typical: “GEP and LogiFlow are the same product.”

You say. GEP is buy + move + plan. LogiFlow is the **move** layer: given constraints, which path. The engineering transfers — ranking on operational data, caching for performance, tests as contracts, and treating the model as one input with guards, not the whole answer. I have not built a purchase-order system. I have built decisioning on operational logistics data.

Q. GEP uses SMART and NEXXE. How would LogiFlow fit?

Typical: “I would replace NEXXE.”

You say. I would not replace anything — I have not seen their internal stack. LogiFlow’s pattern — ingest operational data, rank options under constraints, surface trade-offs to a human — is the same shape as what NEXXE likely does for supply chain visibility. I would learn their system and apply the engineering habits, not transplant my hackathon code.

Q. How would you put AI on procurement data at GEP?

Typical: “RAG, fine-tune GPT, LangGraph agents.”

You say. I have not used LangGraph. Pattern I have used: ingest → validate → model (Gemini categorise on Community Hero, gradient boosting on LogiFlow) → guard → human if unsure. Same shape on invoices or POs: extract fields, never auto-post a bad total. Model is untrusted until validated. I would learn their workflow tool on the job.

7. Hard follow-ups (they drill what you claim)

Q. What is multi-objective ranking? How is it different from sorting?

Typical: “We sort by score.”

You say. Sorting is one dimension — highest score wins. Multi-objective means no single “best” — a fast expensive route and a slow cheap route are both valid. Pareto frontier: I show all non-dominated options. The user picks based on their constraint, not my opinion. If I collapsed to one score, I would hide trade-offs.

Q. What is gradient boosting? Why not a neural network?

Typical: “It is a type of AI.”

You say. Gradient boosting builds trees sequentially — each tree corrects the errors of the previous one. sklearn’s GradientBoostingRegressor. Why not a neural net: 15k records is small for deep learning; boosting handles tabular data well with less tuning. MAE around 22 minutes. I can explain bias-variance if asked, but I will not claim I tuned a transformer on this.

Q. What is MAE 22 minutes? Is that good?

Typical: “It is very accurate.”

You say. MAE means on average the predicted delay is 22 minutes off from actual. For freight routing where delays are measured in hours, 22 minutes is useful as a risk signal — it tells you “this corridor tends to run late.” It is not precise enough to promise a 2:00 PM arrival. I surface it as a risk indicator, not a guarantee.

Q. What if I gave you real-time GPS data? How would you change LogiFlow?

Typical: “I would use deep learning.”

You say. Real-time data changes the caching story — TTL drops, maybe I stream updates instead of cache-aside. The ranking would factor in live delay instead of historical average. I would still keep the model as one input to the ranker, not the sole decision. The architecture shape stays the same; the data freshness changes.

Do NOT say in R2:

- “I led the whole team” — say co-lead, and name what you owned.
- “We used AI / neural network” — say gradient boosting, sklearn, MAE 22 min.
- “LangGraph / Azure / Angular” — you have not used them.
- Numbers you cannot defend — if unsure, say “I would need to check.”

DO say:

- Co-lead. Three pipelines. 580+ corridors. 15k+ train-days. 100–400 ms. pytest 100/100.
- Redis is not the source of truth. SQL is.
- Ranking is the product. The model is one input.
- “I have not used X. The closest I have used is Y.”

8. Quick-fire numbers (memorise these)

Global rank	106 (Google Solution Challenge 2026)
Corridors	580+
Train-day records	15,000+
Response time	100–400 ms (with Redis)
Test suite	100/100 pytest (business rules)
Delay model	Gradient boosting, sklearn, MAE ~22 min
Stack	Python, FastAPI, TypeScript, Next.js, Redis, SQL, GCP Cloud Run
Your pipelines	Railway, Comparator, Hybrid
Role	Technical co-lead
Team size	4